SQL Workshop Speaker Notes

Thank you Eric! Good morning, everyone! Welcome to the BMO Finance Research and Trading Lab. For those I haven't met, my name is Yi-Ming Chang, and I am the Educational Developer at the Lab. It is great to see a full room of Rotman Commerce (Finance, Accounting, and Business) students here on a Saturday.

As Eric mentioned, we will have a lunch break from 12:00 to 1:00 PM, and a shorter break later between 2:30 and 2:45 PM. Please feel free to remind me if I lose track of time when those get close. Also, feel free to raise your hand whenever you have a question or are confused by the topics. Or got stuck with SQL syntax, Mention Hassan.

I want to kick things off with a question. Raise your hand if you have ever tried to open an Excel file that was so large, it completely crashed your computer or couldn’t display fully in Excel.

Today, we are working with a dataset that easily breaks that limit. On the screen behind me is a screenshot of our transactions data table, which is one of the data tables we are going to play around with. It cuts off at exactly 1 million, 48 thousands, 5 hundreds and 76 rows — because that is the absolute maximum number of rows Excel can physically handle. When you hit that wall, you realize you need a better tool. Today, that tool is SQL.

We designed this one-day workshop for you. You don't need have a computer science background, but you do know the value of data in finance and business. Our goal today is not to turn you into data engineers or software engineers. Our goal is to give you a superpower: the ability to pull your own data, exactly how you want it, without ever having to wait on an IT department.

By 4:00 PM today, you will go from having zero coding experience to writing complex queries. We are going to cover the core logic— `SELECT` statement. It’s going to be hands-on, interactive, practical, and hopefully, a lot of fun. Let's get started.

Workshop Overview:

- Introduction to SQL (What, Why, How)
- Relational and Non-relational Data (Database)
- Installation Guide and Recommended Learning Resources (Types of SQL and SQL sub-languages)
- Data Schema: Preview the data types for the tables
- `SELECT` Statement
- `DISTINCT` Clause
- `LIMIT` and `OFFSET` Clause
- `WHERE` Clause with Wildcards
- SQL Built-in Functions (Aggregate Functions, Scalar Functions, and Other Functions)
- Operators (Logical, `LIKE` `IN` `NOT`, etc.)
- `ORDER BY` Clause
- `GROUP BY` and `HAVING`
- SQL JOINs (`INNER JOIN` `LEFT JOIN` `RIGHT JOIN` `FULL OUTER JOIN`)
- We Exercises on HackerRank or LeetCode if time permits
- (Below Not going to cover within this SQL Workshop, maybe in a more advanced SQL workshop)
- Advanced Topics (`CASE` `IF` `VIEWS`), Subqueries and Views
- Operators (UNION UNION ALL EXCEPT INTERSECT
- Indexing and Performance Optimization
- Transactions and Concurrency Control

First, I am going to introduce what SQL is, why we are learning it, and what we can do with it. Then, I will talk about the differences between relational and non-relational data and databases. I will briefly go over the installation process for SQL-related software and their differences, along with the SQL sub-languages.

Later, we will learn the importance of data schemas to preview the datatypes for the tables before we dive into SQL syntax. After that, I am going to cover the `SELECT` statement, `DISTINCT`, `LIMIT`, `OFFSET`, and the `WHERE` clauses with wildcards. Then, we will look at some SQL built-in functions, such as aggregate and scalar functions. I will also talk about some operators in SQL and when to use them.

Next, I will cover the `ORDER BY`, `GROUP BY`, and `HAVING` clauses. Finally, we will talk about `JOIN`s and wrap up with a summary. If time permits, we will do some practice exercises on HackerRank or LeetCode.

What is SQL?

SQL stands for Structural Query Language. It is the standard programming language used to communicate with relational database management systems. You can think of SQL as a specialized translator between you and your data. When you want information—like finding a specific customer or calculating total sales—you cannot simply

speak English to the computer. Instead, you write a query in SQL. The database engine receives this request, processes it, and returns exactly the information you asked for. Unlike general-purpose coding languages like Python or C#, which are used to build models, applications and software, SQL is built for a single, specific purpose: managing and retrieving data efficiently.

Why are we learning SQL?

As I mentioned earlier, imagine trying to open a spreadsheet with over a million rows—Excel simply crashes. In the real financial world, datasets rarely fit neatly into a single spreadsheet. SQL is the fundamental skill that allows you to break through that barrier, letting you handle massive amounts of financial, accounting, and business data efficiently.

Beyond that, it is currently the number one technical skill requested for analyst roles right after Excel. Having SQL on your resume is a critical differentiator. It is the absolute industry standard for the employers you are all targeting. The Big Five Canadian banks, the Big Four accounting firms, and major tech companies all rely on SQL to manage their data ecosystems.

Whether these institutions are using modern cloud platforms like Microsoft Azure and Google Cloud BigQuery, or traditional data warehouses and data lakes, SQL is the universal language used to access that information. By mastering t his skill today, you are essentially future-proofing your career across these top-tier institutions.

So, what can we do with SQL?

First, it empowers you to move beyond simple data entry and into true data analysis. In today's hands-on workshop, you will gain practical experience querying data directly. You'll be able to ask complex business questions and get immediate answers, all without having to submit a ticket to IT.

But beyond direct analysis, SQL is the hidden engine that powers modern business intelligence. Tools like Power BI, Tableau, and Salesforce completely rely on SQL to pull data from databases and feed it into interactive reports. When you learn SQL, you learn how to structure data specifically for these dashboards, which lets you translate complex numbers into clear visual stories. This is how you communicate effectively with stakeholders who might not have your financial or technical background. It ensures they immediately understand the 'so what' behind the data.

Let me show you what that looks like in practice. There are a few dashboards we are going to demonstrate the power of integrating SQL with analytics tools like SaaS (Software as a Service tool).

The screen behind me, we have a Marketing Dashboard. Instead of staring at rows of numbers, you can easily visualize actual financial performance versus the budget, track it monthly or cumulatively, and instantly break it down by department.

Next to it is an IBM HR Dashboard. At a glance, management can track total headcount, monitor the attrition rate, check overtime conditions, and analyze workforce demographics.

Ultimately, these dashboards show how SQL allows you to quickly take millions of rows of unreadable data and transform them into the exact actionable insights that executives use to drive the business forward.

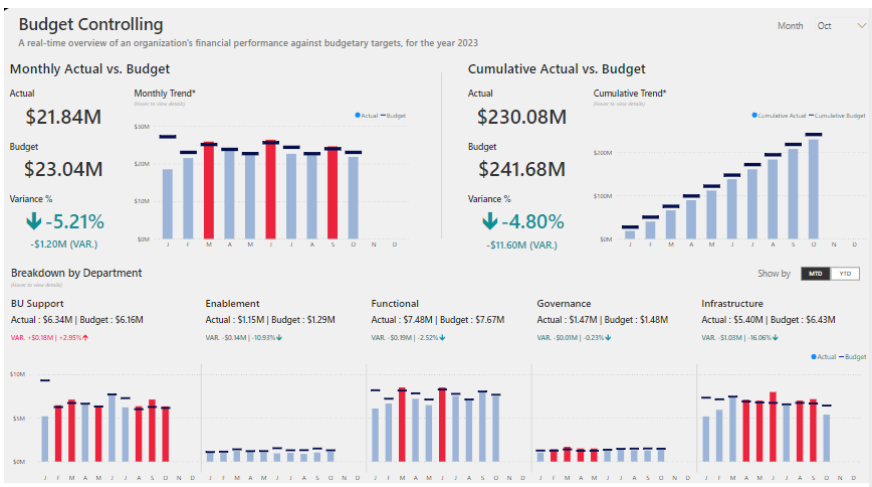


Image retrieve from google image search, Marketing Dashboard example

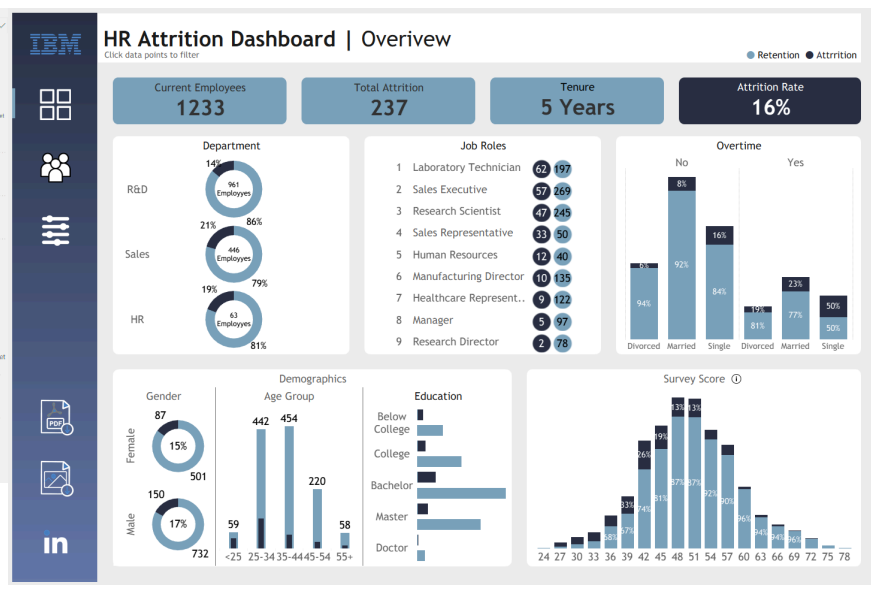


Image retrieve from google image search, IBM Dashboard example

What type of SQL languages and databases are there?

Types of SQL Languages (The "Dialects")

I want to quickly talk about the 'Dialects' of SQL. Technically, there is only one 'Standard SQL.' It is called ANSI SQL, set by the American National Standards Institute. However, different tech companies have taken that standard and created their own proprietary dialects.

The best way to think of this is like the English language. Whether you are in the US, the UK, or Australia, you can easily understand each other. But, you will occasionally run into local slang or spelling differences. SQL works the exact same way. For example, if a company uses Microsoft SQL Server, their dialect is called T-SQL. It has a few special features for handling errors and variables. If a bank is using Oracle databases, their dialect is called PL/SQL, which is designed to handle massive, complex logic. PostgreSQL has its own version called PL/pgSQL.

You may get stress about there are so many different SQL dialects, which one should we learn? Here is the good news: despite these different dialects, roughly 90% of the syntax is completely identical across all of them. The core `SELECT`, `FROM`, and `WHERE` commands we are learning today do not change. So no need to worry about which one you learn. If you know one well, the others are easy to pick up. I believe you can adapt to any company's specific dialect in just a few hours.

Open-Source

Database	Notes	Dialect Example
MySQL	Very popular, web apps, fast reads	MySQL SQL
PostgreSQL	Advanced, highly standards-compliant, loved in data/finance	PostgreSQL SQL
SQLite	Lightweight, file-based, embedded systems	
MariaDB	MySQL fork, better performance in some cases	

Commercial / Enterprise

Database	Notes	Dialect Example
Oracle DB	Extremely powerful, expensive, enterprise	**PL/SQL**
Microsoft SQL Server	Tight Windows + Azure integration	**T-SQL**
IBM Db2	Banking, insurance, mainframes	
SAP HANA	In-memory, analytics-heavy	

What is a Database Management System (DBMS)? The Categories of SQL Commands (Sub-languages)

A database management system (DBMS) is a set of programs used to define, administer, and process databases and their associated applications.

In simple way, a database is essentially a structure you build to hold valuable data. The database management system (DBMS) is the actual software you use to build that structure and interact with the data inside it. There are many on the market. Some run on massive corporate servers, while others run even right on your smartphone. When a DBMS can handle both small and massive workloads, we call it *scalable*.

Today, the biggest trend is hosting these systems in the cloud—whether that is a public cloud managed by Amazon, Google, or Microsoft, or a private cloud secured on a bank's internal network.

If you look at the screenshot image on the screen, you'll see that you—the user—never touch the raw data directly. You communicate with the DBMS, and the DBMS goes into the storage to get what you need. It does the heavy lifting so you don't have to worry about how the files are physically saved on a hard drive.

By the way, image retrieved from “SQL for Dummies”, it is one of the great books I would recommend for beginners to learn SQL.

In the finance world, you will almost exclusively work with Relational Database Management Systems. These are the strict, structured, table-based systems.

Now, how do we actually talk to this DBMS? We use SQL. And SQL is broken down into 5 sub-languages based on what we want to do.

1. **DDL (Data Definition Language)** handles the structure. We use `CREATE` or `DROP` to build or destroy tables.
2. **DML (Data Manipulation Language)** handles the contents. We use `INSERT`, `UPDATE`, or `DELETE` to change the actual data inside the tables.
3. **DCL (Data Control Language)** handles security. IT uses `GRANT` or `REVOKE` to control who can see sensitive financial data.
4. **TCL (Transaction Control Language)** manages the integrity of complex data updates.

5. **DQL (Data Query Language)** is all about retrieving data. This is the `SELECT` command, and it is where we will be spending the rest of our day.

Most of the time, you don't need to worry about the first four sub-languages (DDL, DML, DCL, or TCL). Because you are not **a** data engineer or ITs, you will not be required to create the core data tables, manage server security, or configure system architecture. As business and financial analysts, your primary focus will be almost exclusively on DQL, which is the `SELECT` statement—to retrieve and analyze the data that has already been structured for you.

Figure 1-1 shows that the user communicates with the database through the DBMS. The DBMS masks the physical details of the database storage so that the application need only concern itself with the logical characteristics of the data, not with how the data is stored.

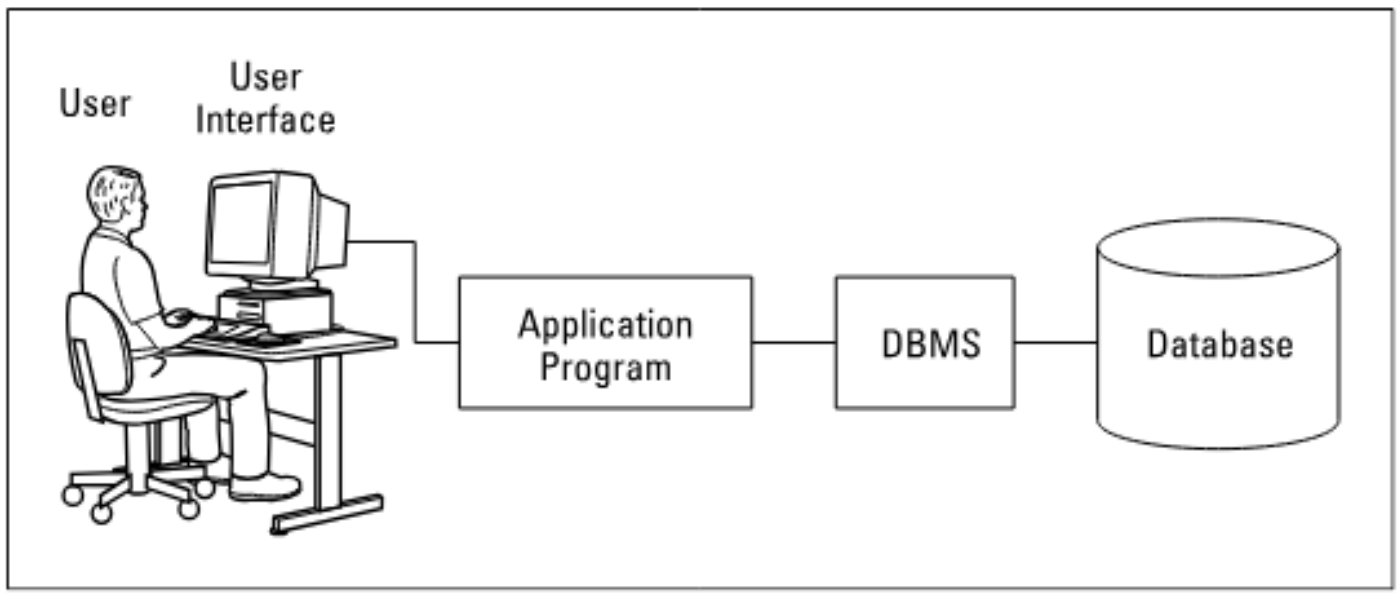


FIGURE 1-1: A block diagram of a DBMS-based information system.

Image retrieved from "SQL for Dummies, Allen G. Talyor (2018, Wiley), 9th Edition."

So, what exactly is data (What are relational and non-relational data)?

Data is simply recorded information, but how we organize that information changes and how we use it.

Let's start with Relational Data. This is what you will see 95% of the time in finance, accounting and operational business systems. It is highly organized and structured, just like a neatly kept filing cabinet or a collection of Excel spreadsheets. In a relational database (like SQL Server or Oracle), everything is stored in tables with fixed rows and columns, and every piece of data has to follow strict rules. A 'Price' column can only hold numbers. These tables are then linked together using unique keys (like your student ID). This rigid structure guarantees accuracy and consistency, which is exactly why banks rely on it. In a banking system, a misplaced decimal point could be a disaster!

In the modern financial world, you will frequently hear about platforms like Snowflake, Google BigQuery, and Amazon Redshift (Cloud Data Warehouses). These are specialized SQL databases designed strictly for analytics (not daily transactions). They replace old-school servers because they separate "Storage" from "Computing Power." This allows banks to scan billions of rows in seconds and scale up their power only when running heavy end-of-month reports.

Non-Relational & Cloud Databases

Unlike strict SQL tables, Non-Relational databases are designed for the messy, unstructured data of the internet age. It is also called NoSQL. Imagine a laundry basket where you can throw in anything: PDFs documents, word documents, emails, or social media posts, without needing to fold or organize them first. NoSQL database prioritize flexibility and speed over rigid structure. For example, one customer profile might just have a name, while another has a name, three profile photos, and a history of website clicks.

Think about your media posts. Have you ever wonder how your Instagram posts are actually stored? Or when you do a Google Image search, where are all those pictures instantly coming from? Think about YouTube: how is a massive video file stored, and how does the system instantly link it to millions of live comments? (The truth is, nearly everything you see through your eyes can be digitized. Even the human body can be mapped, converted into raw data, and stored in a database somewhere in the world.)

Let's look at Instagram as an example. When you upload a photo, you are not stuffing a heavy image file into a relational database table cell. That actual `.jpg` or `.png` file goes into a massive digital warehouse called Object Storage (like an infinite hard drive). Your database just holds a tiny text link pointing to that image (like `instagram.com/user/photo_123.jpg`). So, when you open your profile, the app asks the database, 'Give me the list of links for Yi-Ming.' The database instantly hands over the links, and your phone loads the pictures. That separation is what makes these apps so incredibly fast.

You see the same thing with YouTube videos, where videos are binary data (BLOBs). It is massive and unstructured. Obviously, you cannot put a 1GB movie file into an Excel cell. Think of a YouTube video page. One video has a Title, Description, and Tags. Another video has all that *plus* Subtitles in 40 languages, Chapter markers. A stream of millions of comments, some with replies, some with images, some deleted. (or searching through a decade's worth of dense, text-heavy legal contracts). A traditional SQL database is terrible at reading paragraphs, while, a non-relational Document database can search through millions of words instantly, just like Google Search.

Another example is the Graph Database. And to be clear, we don't mean bar charts or line graphs; we mean a spiderweb of connections. (Imagine a criminal uses a stolen credit card. That is Node A. They ship the item to an address—Node B. A Graph database instantly 'walks' the spiderweb and realizes that Node B was used three years ago by a known fraudster—Node C. It lights up that hidden connection and declines the transaction before the money is ever stolen.)

The common NoSQL Types are:

- Common NoSQL examples (not SQL): MongoDB, Redis, Neo4j, Cassandra, DynamoDB.
- Document Stores (e.g., MongoDB): Stores data in flexible "JSON" documents (like a nested list). Great for product catalogs.
- Key-Value Stores (e.g., Redis): Extremely fast simple storage. Used for caching (remembering your login session).
- Graph Databases (e.g., Neo4j): Focuses on connections. Used for social networks and fraud detection (finding hidden links between criminals).

Don’t mention: Social Media & Images (Instagram, Google Photos)

How an app like **Instagram** shows you *your* specific photos when you open your profile.

When you upload a photo to Instagram, two things happen:

1. **The Image File (The Heavy Lifting):** The actual picture (the .jpg or .png) is sent to a massive digital warehouse called **Object Storage** (like an infinite hard drive). It is **not** stored in a database table because it is too big.
2. **The Link (The Reference):** The database stores a "pointer"—basically a text link (like `instagram.com/user/photo_123.jpg`).

When you open your profile: The app asks the database: *"Give me the list of links for User ID: Yi-Ming."* The database instantly returns the list of links, and your phone downloads the images from the warehouse to display them. This separation is why non-relational systems are fast—they don't try to stuff the heavy image into the rows and columns.

Videos & Streaming (YouTube, Netflix)

- **The Data:**
  - **The Video File itself:** This is binary data (BLOBs). It is massive and unstructured. You cannot put a 1GB movie file into an Excel cell.
  - **The Metadata:** Think of a YouTube video page. One video has a Title, Description, and Tags. Another video has all that *plus* Subtitles in 40 languages, Chapter markers, and a "Merchandise Shelf."
  - **The Comments:** A stream of millions of comments, some with replies, some with images, some deleted.
- Video metadata varies wildly. Storing the *stream* of millions of likes/views per second requires the speed of a non-relational database (like **Cassandra** or **BigTable**) rather than the strict rules of SQL.

Documents (PDFs, Word Docs)

When we say "Document Database" (like MongoDB), we usually mean **JSON data** (text code), but non-relational databases are also used to manage actual user files like PDFs. Think of legal contracts from a law firm, medical history scans, or resumes. These are full of text, paragraphs, and sentences. Imagine you are a lawyer and you need to find every contract from the last 10 years that mentions the phrase *"Force Majeure"* or *"Bankruptcy."* SQL databases are terrible at reading paragraphs; they are built to match exact numbers (Price = $100). Non-relational "Document Stores" (like **MongoDB** or **Elasticsearch**) break down the PDF into individual words. This allows you to search inside the documents instantly, exactly like how Google Search works.

Graphs (Networks, NOT Charts)

When we say "Graph Database" (like **Neo4j**), we do not mean bar charts or line graphs. We mean a Spiderweb of connections. A Graph database is built to "walk" the spiderweb instantly to find the connection. A Graph database lights up these hidden links immediately, flagging the transaction before the money is stolen. A criminal uses a stolen card (Node A) but uses the same shipping address (Node B) as a previous known fraudster (Node C).

Summary Checklist

If the data looks like...	It is likely...	Example
A neat Grid	**Relational** (SQL)	Bank Transactions, Student Grades
A Tree / Nested List	**Document** (NoSQL)	Shopping Cart, User Profile
A Spiderweb	**Graph** (NoSQL)	Social Network, Fraud Ring
A Firehose	**Key-Value / Wide-Column**	Stock Ticker Feed, IoT Sensors

Feature	Relational (SQL)	Non-Relational (NoSQL)
Data Shape	**Tables** (Rows & Columns). Looks like a spreadsheet.	**Documents, Key-Values, Graphs**. Looks like a JSON file or a tree.
Flexibility	**Rigid.** You must define the structure (schema) *before* adding data. Changing it later is hard.	**Flexible.** You can add new fields on the fly without breaking the system.
Relationships	**Strong.** Tables are linked (e.g., Student ID connects to Grades table).	**Weak/None.** Data is often self-contained.
Scaling	**Vertical.** To handle more data, you need a *bigger* computer (more RAM/CPU).	**Horizontal.** To handle more data, you just add *more* cheap computers (servers).
Transactions	**ACID Compliant.** Guaranteed accuracy. (If money leaves Account A, it *must* arrive in Account B).	**BASE Model.** "Eventually Consistent." (It's okay if a "Like" count is off by 1 for a few seconds).

Installation of SQL software or database management tool

There are many open-source database systems available, such as MySQL, PostgreSQL, and Microsoft SQL Server Management Studio. Today, we will be using DBeaver. Please note that DBeaver is a *Database Management Tool* (GUI), not a database itself. It provides a universal interface that allows you to connect to and interact with various database systems, including MySQL and PostgreSQL, even NoSQL database, but these features will required to be paid.

Suggested SQL Textbooks: “SQL for Dummies” by Allen G. Taylor, and “Learning SQL” by Alan Beaulieu.

Suggested self-learning materials website: W3School SQL Tutorial, (Advanced) GeeksforGeeks SQL Tutorial.

Exercise: LeetCode, HackerRank (IBM, Capital One, Goldman Sachs), or ask Gen AI to create some questions

The Data Schema

Before we start, I’d like to introduce the concept of a Data Schema. A schema is essentially the blueprint or map of your database. It defines the structure of your data, including the relationships between tables, fields, and indexes. Think of it this way: Just like you wouldn't try to build a house without a blueprint, or navigate a maze without a map. Imagine walking into a massive library with millions of books but no signs or catalog. It would be hard for you to find the book you want. Therefore, you cannot effectively query a database without understanding its schema.

A schema tells you data types upfront. It also reveals the "Hidden Connections": This is the most important part for SQL Joins. Data is rarely in one big table; it is split up to be efficient (Normalization).

Here is an example of data schema for a Retail Sales DB. This schema represents a standard Transactional Sales System. It allows us to track *who* bought something (from Customers table), *what* they bought (from Products table), *who* sold it to them (from Employees table), and *when* it happened (from Orders table).

Entity Relationships

This schema follows a Star Schema logic, where the `Orders` table sits in the middle, called child table, and the other tables surround it, called parent tables. A parent table is a primary table containing a unique identifier (primary key) in a database, while a child table contains a corresponding foreign key that references the parent. When a primary key appears in another table, that field is called a foreign key in that table. The relationship between primary keys and foreign keys creates the relationship between tables.

Let's take a look into the details of these tables. Customers table has data columns: customerID, first name, last name, birth data, money spent and anniversary. The corresponding datatype are integers, string, string, data, money and date.

Money Datatype meant decimal datatype, which is Numeric type used for currency to ensure exact precision.

DATE: Stores calendar dates (YYYY-MM-DD).

As you can see, the diagram uses color-coded lines to explicitly map the foreign key relationships to the central `Orders` table

- An orange line connects the `customerID` in the Customers table to the `customerID` in the Orders table. It tells us exactly who made the purchase.
- A purple line connects the `employeeID` in the Employees table to the `employeeID` in the Orders table. It tells us who facilitated the sale, which is crucial for calculating sales commissions and tracking staff performance
- A blue line connects the `productID` in the Products table to the `productID` in the Orders table. It tells us exactly what was sold, linking the transaction back to our inventory and pricing data.

Because of these relationships, we can track the entire lifecycle of a sale. We can see how many orders a customer made, what products they bought, and exactly which employee helped them.

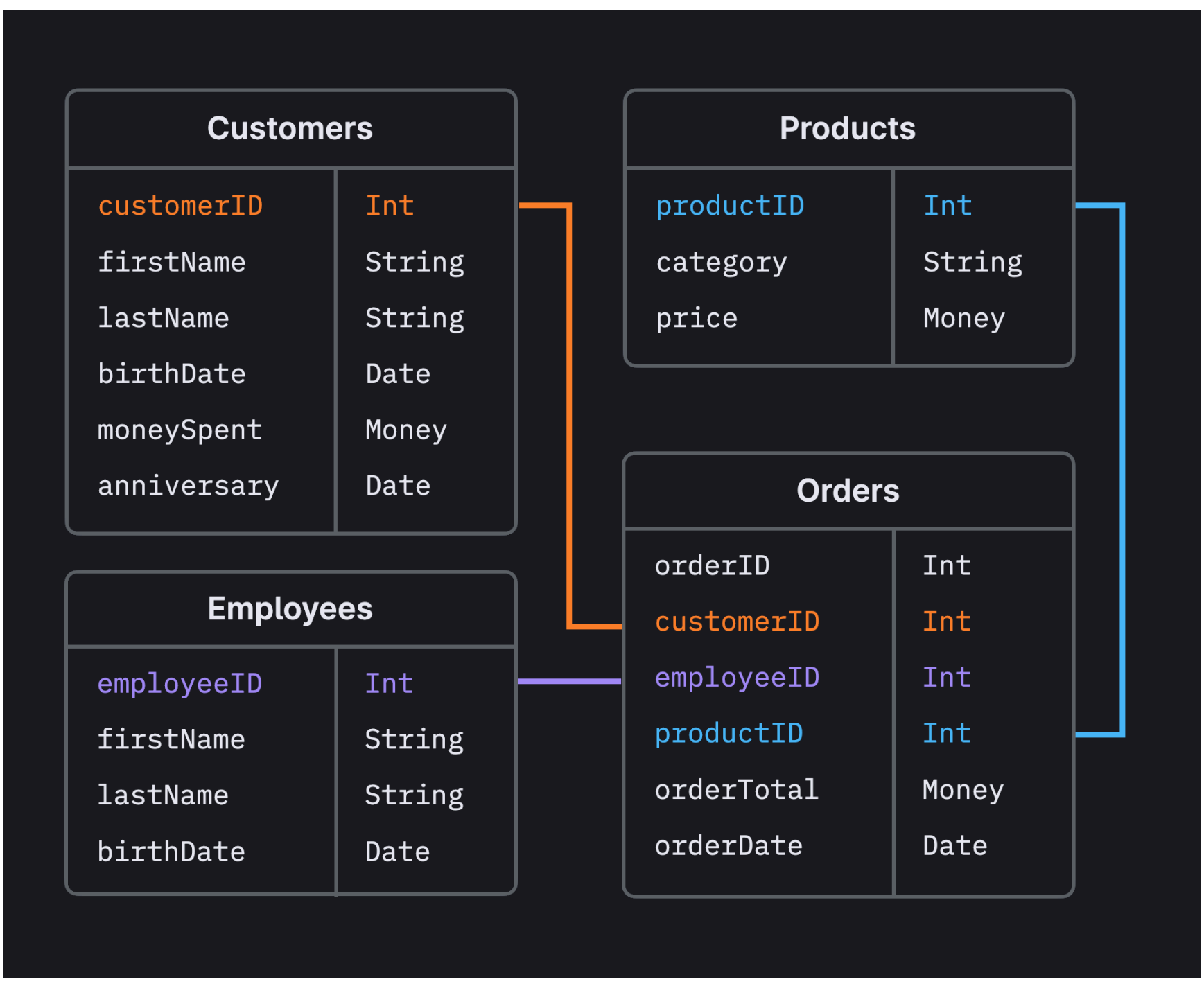


Image retrieved from schema-design-101-relational-databases.

1. Table: Customers

3. Table: Employees

Stores the personal profiles and spending history of clients.

Column Name	Data Type	Description
`customerID`	`INT`	**Primary Key.** Unique identifier for each customer.
`firstname`	`VARCHAR`	The customer's first name.
`lastname`	`VARCHAR`	The customer's last name.
`birthdate`	`DATE`	Used for age demographics or birthday marketing.
`moneySpent`	`DECIMAL`	A running total of lifetime spending by this customer.
`anniversary`	`DATE`	Date of joining the loyalty program or first purchase.

2. Table: Products

The catalog of items available for sale.

Column Name	Data Type	Description
`productID`	`INT`	**Primary Key.** Unique identifier for the product.
`category`	`VARCHAR`	The grouping of the item (e.g., 'Electronics', 'Clothing').
`price`	`DECIMAL`	The individual cost of the item.

The staff members responsible for processing orders.

Column Name	Data Type	Description
`employeeID`	`INT`	**Primary Key.** Unique identifier for the staff member.
`firstname`	`VARCHAR`	The employee's first name.
`lastname`	`VARCHAR`	The employee's last name.
`birthdate`	`DATE`	The employee's date of birth (used for HR/Age requirements).

4. Table: Orders

The central log of all transactions. This is the "Fact Table" that connects everything else.

Column Name	Data Type	Description
`orderid`	`INT`	**Primary Key.** Unique ID for the specific receipt/transaction.
`customerid`	`INT`	**Foreign Key.** Links to `customers.customerID`. Identifies who bought it.
`employeeID`	`INT`	**Foreign Key.** Links to `Employees.employeeID`. Identifies who sold it.
`productID`	`INT`	**Foreign Key.** Links to `products.productID`. Identifies what was sold.
`ordertotal`	`DECIMAL`	The final dollar amount of the transaction.
`orderdate`	`DATE`	The date the transaction occurred.

Kaggle Financial Transactions Fraud Datasets (Create from SQL database `.mdf` file + LINQ to SQL database `dbml` file)

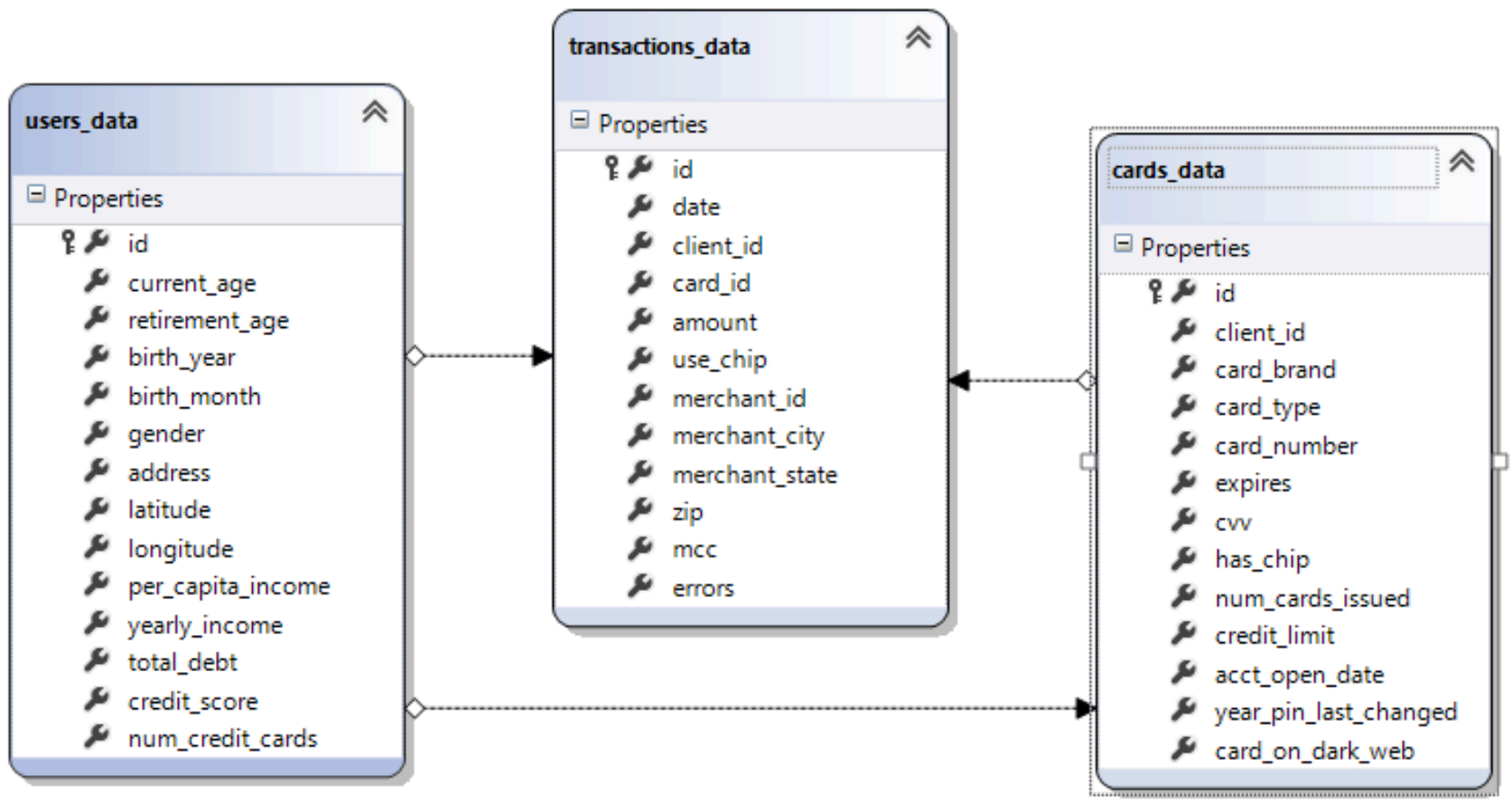
The dataset we are using for this SQL workshop is sourced from Kaggle, specifically called Financial Transactions Fraud Datasets: Kaggle-Transaction-Fraud-Datasets. The database is divided into three distinct tables, each serving a specific purpose:

- The `users_data` table is the Client Profile. This is a demographic table. Every single row represents one unique person. It holds static or slowly changing information like their age, location, yearly income, and credit score.
- The `cards_data` table represents the bank's Financial Products. This tracks the physical (or digital) credit or debit cards, including the brand, the credit limit, expiration date, etc.
- The `transactions_data` table acts as the Action Ledger. Every time a card is swiped, a new row is recorded here with the date, amount, and merchant, etc. Remember our Excel limits? This table alone has over 13 million rows. That is exactly today why you need SQL to read the data table today.

Each table has a Primary Key, indicated by the little key icon next to the `id` column. This guarantees that every user, card, and transaction has a 100% unique identifier. To link everything together, the `client_id` and `card_id` in transactions table act as Foreign Keys that connect back to the `id` columns of `users` and `cards` data tables. Because of these relationships, we can track exactly how many transactions a user has made, which specific card they used, where the transaction took place, and for what amount—especially since a single customer can hold multiple cards. Ultimately, this data tells us the complete story of their financial behavior and their preferences (after we done some financial analysis on the dataset).

Now open the DBMS tool (`DBeaver`) on the lab computers, setup environment to practice, we then can start our hands-on practice. (Around 40-45 mins)

Click on New Database Connection → Select SQLite → Pick URL click finish → then import data to our created database → Click on SQL Edit create new SQL script. (Around ?? mins)



1. `users_data` **(The Customers)**
   - **Primary Key:** `id`
   - **Key Columns:** `current_age`, `yearly_income`, `credit_score`, `total_debt`.

Column Name	Data Type	Description
`id`	`INTEGER`	**Primary Key.** Unique ID for the customer, client id
`current_age`	`INTEGER`	Customer's age.
`retirement_age`	`INTEGER`	Expected retirement age.
`birth_year`	`INTEGER`	Year of birth.
`birth_month`	`INTEGER`	Month of birth.
`gender`	`TEXT`	Gender (Male/Female).
`address`	`TEXT`	Street address.
`latitude`	`FLOAT`	Geographic latitude of residence.
`longitude`	`FLOAT`	Geographic longitude of residence.
`per_capita_income`	`TEXT`	Area income level (Contains '$' symbol, e.g., `$29278`).

2. `cards_data` **(The Products)**
   - **Primary Key:** `id`
   - **Foreign Key:** `client_id` (Links to `users_data.id`)
   - **Key Columns:** `card_brand`, `card_type`, `credit_limit`.

Column Name	Data Type	Description
`id`	`INTEGER`	**Primary Key.** Unique ID for the specific card.
`client_id`	`INTEGER`	**Foreign Key.** Links to `users_data.id`.
`card_brand`	`TEXT`	Brand of the card (Visa, Mastercard, Discover, Amex).
`card_type`	`TEXT`	Type of card (Credit, Debit, Debit (Prepaid)).
`card_number`	`FLOAT`	Masked card number.
`expires`	`TEXT`	Expiration date (MM/YYYY).
`cvv`	`INTEGER`	Security code.
`has_chip`	`TEXT`	Does the card have a chip? (YES/NO).
`num_cards_issued`	`INTEGER`	Number of replacement cards issued.
`credit_limit`	`TEXT`	Max spending limit (Contains '$' symbol).

3. `transactions_data` **(The Behavior)**
   - **Foreign Key:** `card_id` (Links to `cards_data.id`)
   - **Key Columns:** `amount`, `date`, `merchant_city`, `mcc` (Merchant Category Code).

Column Name	Data Type	Description
`id`	`INTEGER`	**Primary Key.** Unique ID for the transaction.
`card_id`	`INTEGER`	**Foreign Key.** Links to `cards_data.id`.
`date`	`DATETIME`	Timestamp of the transaction.
`amount`	`TEXT/FLOAT`	Value of the transaction (Check for '$').
`use_chip`	`TEXT`	Method used (Swipe, Chip, Online).
`merchant_id`	`INTEGER`	ID of the merchant.
`merchant_city`	`TEXT`	City where transaction occurred.
`merchant_state`	`TEXT`	State/Region of merchant.
`zip`	`FLOAT`	Zip code of merchant.
`mcc`	`INTEGER`	**Merchant Category Code** (e.g., 5411 for Grocery).
`acct_open_date`	`TEXT`	Date account was opened (MM/YYYY).
`year_pin_last_changed`	`INTEGER`	Year the PIN was last updated.
`card_on_dark_web`	`TEXT`	Security flag (No/Yes). refers to stolen credit or debit card information

For a complete list of functions supported by a specific database system, refer to its official documentation, such as the Microsoft Learn SQL documentation or the SQLite Built-In Scalar SQL Functions documentation.

The `SELECT` statement is used to select data from a database.

Goal: Know Your Customer. Understanding the demographics.

Files used: `users_data`

Unicode is an international computing standard that assigns a unique numeric **code point** to every character, symbol, and emoji across all writing systems, ensuring consistent text representation regardless of platform or language. It enables seamless digital communication by replacing older, limited encoding systems (like ASCII) with a universal, scalable standard.

- **Universal Coverage:** It supports over 150,000 characters, including modern, historical, and technical scripts.
- **Code Points:** Characters are assigned unique, standardized numbers (e.g., `U+0041` for 'A').
- **Encoding Schemes:** Unicode code points are mapped to bytes using encodings like **UTF-8** (most common for web) or **UTF-16**.

Difference from ASCII: ASCII is limited to 128 characters (English), while Unicode covers nearly all languages.

Purpose: It prevents text corruption when transferring data between different systems or software.

Unicode is maintained by the Unicode Consortium to ensure cross-platform compatibility.

Difference between VARCHAR(50) and NVARCHAR(50):

The core difference between `VARCHAR` and `NVARCHAR` comes down to Language and Storage Space. The "N" in `NVARCHAR` stands for National (or Unicode). It tells the database to prepare to store characters from *any* language in the world, not just standard English letters.

1. `VARCHAR(50)` (Variable Character)

- **What it stores:** Standard ASCII characters (American Standard Code for Information Interchange, English letters, numbers, and basic symbols).
- **Storage size:** 1 byte per character.
- **Max Capacity in this example:** 50 characters = **50 bytes**.
- **What breaks it:** If someone tries to insert Japanese Kanji, Arabic script, or even certain emojis, the database might turn them into question marks (`???`) because it doesn't know how to read them.

2. `NVARCHAR(50)` (National Variable Character)

- **What it stores:** Unicode characters. It can store English, plus Russian, Chinese, Hindi, emojis (😎), and special characters.
- **Storage size:** 2 bytes per character (because it needs more memory to map all the world's alphabets).
- **Max Capacity in this example:** 50 characters = **100 bytes**.
- **The Trade-off:** It is safer for international data, but it takes up exactly **twice as much storage space** on your hard drive.

Let's look into the data type in each table before we start. First, I would like to teach you PRAGMA. We can use PRAGMA to preview the data types for the tables `PRAGMA table_info(table_name);` press CTRL + ENTER will run the specific block of SQL code in DBeaver. And remember always end with a semi-colon (;) after you complete a query.

- cid are just indexes from DBeaver for you to index the columns of a table.
- Name: the name of each column
- Type: datatype stored in each column
- notnull: this tells you whether a column is allowed to be empty (NULL). 0 = False, can be empty; 1 = True, cannot be empty
- dflt_value: Default value. It shows what the database will automatically insert if a user creates a new row but forgets to provide data for this specific column. If it is blank (NULL): There is no default. If it has a value: That exact value gets inserted automatically.
- pk (primary key): 0 is the column is not primary key; 1 is the column is primary key.
- Just for your information the id columns should be 1 (Note: client_id, card_id, and transaction_id). However, since we downloaded this raw data from Kaggle, these columns were not pre-configured as Primary Keys. But they are actually primary key of the tables, where the values are unique.

(Then explain the datatype labels meaning, `VARCHAR` `NVARCHAR` `INT` `BIGINT` `REAL` `DECIMAL` `FLOAT`

`VARCHAR` is just "String" (Text datatype), called variable character. Variable character 50, means maximum 50 characters are allow, same with `NVARCHAR(50)` is text datatype, called national variable character. The core difference between `VARCHAR` and `NVARCHAR` comes down to Language and Storage Space. The "N" in `NVARCHAR` stands for National (or Unicode). It tells the database to prepare to store characters from *any* language in the world, not just standard English letters. It can store English, plus Russian, Chinese, Hindi, emojis (😎), and special characters.

`Real` number datatype: rational and irrational numbers, exclude complex numbers.

```
# CTRL + ENTER will run the specific block of SQL code in DBbeaver.
PRAGMA table_info(table_name); # we use pragma to preview the datatypes for the table
PRAGMA table_info(Album); # If we preview the datatypes for Album table.

# Due to the complexity of three tables we have. Let's start with the simplest of the
three datasets we are covering today: users_data.
# The simplest SQL query is to select all of the columns. Here we query all of the dat
a from the users_data table.
SELECT * FROM users_data; # note unlike python, SQL is NOT case-sensitive for keyword
s.
seleCT * FrOm Users_DAta; # The database managemet tool treats these two queries exact
ly the same
# You can call out different columns from the table within the database by the field n
ames (column name) of the table.
SELECT id, current_age, birth_year, gender, yearly_income, total_debt
FROM users_data;

SELECT column1, column2, ...
FROM table_name; # Free feel to practice yourself, but don't worry on making your own
notes. I have a SQL basic Syntax template at the end, you can scan the QR code and tak
e away the slides.
```

```
# Next, I would like to show you that we can do some simple calculation ourselves, suc
h as calculate the Debt-to-Income ratio for each customer. It will provide new data co
lumn named exactly total_debt / yearly_income.
SELECT id, (total_debt / yearly_income) FROM users_data;
SELECT id, (total_debt / yearly_income) AS debt_to_income_ratio from users_data; # Whe
re we can rename the data column into whatever name we want. Note, just like Python, w
here you have to include underscores for naming.
SELECT id, (total_debt / yearly_income) AS 'debt_to_income_ratio' from users_data; # o
r you can use single or double qoutation mark for the string.
# Note, when things come with naming, the case of characters is important.

# There are initially 2000 rows display, why happened if we only want to display top 1
0 rows.
SELECT * FROM users_data LIMIT 10; # select top 10
```

Now you have a basic idea of how we query data using SQL, I would like to go over some basic SQL built-in functions.

Aggregate Functions: Aggregate functions operate on a set or a group of values (usually a column) and return a single value. They are commonly used with the `GROUP BY` clause.

- `AVG()`: Calculates the average value of a numeric column.
- `COUNT()`: Counts the number of rows that match a specified criterion/standards. `COUNT(*)` counts all rows, including NULLs.
- `MAX()`: Retrieves the maximum value from a column.
- `MIN()`: Retrieves the minimum value from a column.
- `SUM()`: Calculates the total sum of a numeric column.
- `FIRST()` / `LAST()`: Returns the first or last value in an ordered set of values (availability can depend on the specific SQL dialect). Don't mention it

String Functions: These functions are used to manipulate character data.

- `CONCAT()`: Joins two or more strings together.
- `LENGTH()` / `LEN()`: Returns the length of a string.
- `UPPER()` / `UCASE()`: Converts a string to uppercase.
- `LOWER()` / `LCASE()`: Converts a string to lowercase.
- `SUBSTRING()` / `SUBSTR()` / `MID()`: Extracts a part of a string (a substring).
- `REPLACE()`: Replaces all occurrences of a substring within a string.
- `TRIM()` / `LTRIM()` / `RTRIM()`: Removes leading, trailing, or both leading and trailing whitespace or other specified characters.

Numeric/Mathematical Functions: These functions perform mathematical operations.

- `ABS()`: Returns the absolute (positive) value of a number.
- `ROUND()`: Rounds a number to a specified number of decimal places.
- `CEIL()` / `CEILING()`: Rounds a number up to the nearest integer.
- `FLOOR()`: Rounds a number down to the nearest integer.
- `SQRT()`: Calculates the square root of a number.
- `RAND()` / `RANDOM()`: Generates a random number.

There are more SQL built-in functions, but we won't be covered all of them within this SQL workshop. We will go over most of above functions along the workshop.

Date and Time Functions (**don't include**)

These functions work with date and time data types.

- **`NOW()` / `GETDATE()` / `CURRENT_TIMESTAMP`**: Returns the current system date and time.
- `DATE()`: Extracts the date part of a date/time expression.

- `YEAR()` / `MONTH()` / `DAY()`: Extracts the specific year, month, or day component from a date value.

```
# MySQL version
SELECT *
FROM transactions_data
WHERE YEAR(date) = 2010;

# SQLite version
SELECT *
FROM transactions_data
WHERE strftime('%Y', date) = '2010';

SELECT *
FROM transactions_data
WHERE strftime('%m', date) = '02'; -- For February
```

Aggregate functions operate on a set or a group of values (usually a column) and return a single value. They are commonly used with the `GROUP BY` clause.

- `AVG()` `COUNT()` `MAX()` `MIN()` `SUM()` `FIRST()` `LAST()`

```
-- Let's find the age of our oldest customer.
-- The MAX() function scans the entire column and returns the highest value.
SELECT id, MAX(current_age), birth_year, birth_month, gender
FROM users_data;

SELECT id, MIN(current_age), birth_year, birth_month, gender
FROM users_data;

-- What is the average customer age within the system?
-- The AVG() function calculates the mathematical mean of the column.
SELECT AVG(current_age)
FROM users_data;

-- Average yearly income across of all customer within the users_data table.
SELECT AVG(yearly_income)
FROM users_data;

-- We can calculate multiple metrics at the exact same time.
-- What is the average credit score of our entire customer base, and what is the highe
st debt?
SELECT AVG(credit_score), MAX(total_debt)
FROM users_data;
-- The output columns looked messy (like "AVG(credit_score)").
-- We use the 'AS' keyword to rename the output columns so they are clean and readabl
e.
SELECT
        AVG(credit_score) AS average_credit_score,
        MAX(total_debt) AS maximum_total_debt
FROM users_data;

# Let's introduce another aggregate functions.
-- How many total rows (customers) do we have in the users_data table? (Exactly 2000)
SELECT count(*)
FROM users_data;
-- Let's validate how the AVG() function actually works behind the scenes!
-- An average is simply the total sum divided by the total count.
-- Here, we show that using AVG() gives the exact same result as doing the math manual
ly.
SELECT
```

```
        AVG(yearly_income) AS average_income,
        SUM(yearly_income)/count(yearly_income) AS Average_yearly_income
FROM users_data;

-- Going back to what we discussed earlier, here is how we limit our view to just the
top 10 rows.
SELECT * FROM users_data LIMIT 10; # select top 10
-- What if we want to see the LAST 10 rows of our 2,000-row dataset?
-- The OFFSET command tells the database to skip the first 1990 rows, and then the LIM
IT grabs the next 10 rows (rows 1991 through 2000).
SELECT * FROM users_data LIMIT 10 OFFSET 1990; # select the last 10 rows within the da
taset. 2000 rows
```

How do you handle duplicate values or NULL values in your dataset?

These questions are often to ask during interviews. `count(*)` will count all of the rows in the table, however, it will also include NULL values.

```
-- The 'id' column is the Primary Key, as we mentioned every ID is unique and no dupli
cates are allowed.
-- This query counts all the rows within the 'id' column that are not NULL.
SELECT COUNT(id)
FROM users_data;

-- Because the nature of Primary Key, there are no duplicates.
-- Therefore, adding DISTINCT should return the exact same number as before.
-- This is a great way to "test" your data to prove there are no duplicate accounts!
SELECT COUNT(DISTINCT id)
FROM users_data;

-- If we select just the gender column, we will get a very long list with thousands of
repeating values (e.g., Male, Female, Female, Male...).
SELECT gender
FROM users_data;

-- The SELECT DISTINCT statement filters out all the repetitions.
-- It returns only the unique categories that exist in this column.
SELECT DISTINCT gender
FROM users_data;

-- This counts every single row that has a gender listed (Total population).
SELECT COUNT(gender)
FROM users_data;

-- Database accessment
-- This counts how many UNIQUE categories exist in the column.
-- For this dataset, it will simply return '2' (Female and Male). This show you eactl
y, how to exclude duplicate values and count how many unique categories exist in the c
olumn
SELECT COUNT(DISTINCT gender)
FROM users_data;

##############################
# No need to talk about the following, unless students are wondering.
# What happen if you don't want to include zero values?
SELECT COUNT(total_debt)
FROM users_data
WHERE total_debt != 0; -- Removes rows where debt is 0

SELECT COUNT(NULLIF(total_debt, 0))
```

```
FROM users_data;

-- What happen if you have NULL values? The NULL values indicate the total absence of
data. It means the value is unknown, missing, or was never entered into the database.
-- Let's if there is any missing address from our customer profile.
-- This query returns only the customers who have an actual address on file.
SELECT id, Address
FROM users_data
WHERE Address IS NOT NULL;

-- CRITICAL CONCEPT: NULL is not the same as an empty string ("").
-- NULL means "missing," but "" means the user intentionally entered a blank space.
-- This query safely filters out BOTH completely missing addresses (NULL)
-- AND blank addresses ('').
SELECT id, Address
FROM users_data
WHERE Address IS NOT NULL AND Address IS NOT ""; # is not empty
```

Numeric/Mathematical Functions:

These functions perform mathematical operations.

- `ABS()` `ROUND()` `CEILING()` `FLOOR()` `SQRT()` `RANDOM()`

```
-- Note: SQL Built-in mathematical functions can only be applied to columns with numer
ic data types.
-- 1. THE ABS() FUNCTION: Absolute Value
-- This will return an error because you cannot apply math to an entire row (*) or non
-numeric columns.
SELECT ABS(*)
FROM users_data; # will return error

PRAGMA table_info(users_data);

-- This works perfectly with longitude, as the column is in numeric data type. It retu
rns the absolute value, converting any negative value.
SELECT longitude, ABS(Longitude)
FROM users_data;

-- 2. THE ROUND() FUNCTION
-- This allows you to specify exactly how many decimal places you want to keep.
-- Here, we round the Longitude column to 1 decimal place.
SELECT Longitude, ROUND(Longitude,1)
FROM users_data;

-- 3. THE CEILING() FUNCTION: Always Round Up
-- Unlike the ROUND function, CEILING does not let you choose the number of decimal pl
aces.
-- It ALWAYS rounds a number UP to the next highest whole integer.
SELECT longitude, CEILING(Longitude)
FROM users_data;

SELECT ABS(longitude), CEILING(ABS(longitude))
FROM users_data;
-- Example: Even though it is only .1, CEILING pushes it up to 1001.
SELECT CEILING(1000.1);

-- 4. THE FLOOR() FUNCTION: Always Round Down
-- In contrast to CEILING, FLOOR ALWAYS rounds a number DOWN to the nearest whole inte
ger.
-- Example: Even though it is .7, FLOOR drops the decimal and leaves it at 1000.
```

```
SELECT FLOOR(1000.7);

SELECT ABS(longitude), CEILING(ABS(longitude)), FLOOR(ABS(longitude))
FROM users_data;

-- 5. THE SQRT() FUNCTION: Square Root
-- Returns the square root of a given positive number. Result here is 100.
SELECT SQRT(10000);
-- 6. THE RANDOM() FUNCTION
-- Generates a raw, random number. In many databases, this is a very long decimal or a
huge integer.
SELECT RANDOM(); -- Around 40 mins.

######## No need to mention
-- ADVANCED APPLICATION: Generating a random number within a specific range.
-- This uses ABS() to ensure the random number is positive, and the modulo operator
(%)
-- to cap the result, effectively generating a random integer between 1 and 10.
SELECT
    ABS(RANDOM()) % 10 + 1 AS Random_Number
FROM transactions_data
LIMIT 5;
```

Another Group of SQL Built-in function: String Functions: These functions are used to manipulate character data.

- `CONCAT()` `LENGTH()` `UPPER()` `LOWER()` `SUBSTR()` `REPLACE` `TRIM()` / `LTRIM()` / `RTRIM()` `CAST()` `COALESCE()`: Returns the first non-null value from a list of arguments.

```
-- The CONCAT() function joins two or more strings together into one continuous strin
g.
SELECT CONCAT('A', 'B', 'C') # normally, we write the sql query in this way, but SQLit
e very recent updates does not have a built-in CONCAT() function like MySQL or SQL Ser
ver. Instead, SQLite uses the standard double pipe operator (||) to glue strings toget
her.
SELECT 'A' || 'B' || 'C' AS concatenated_string;

-- Real-world example: Combining address and GPS coordinates into one readable column.
SELECT address || latitude || longitude AS address_with_coordinates
FROM users_data;
SELECT CONCAT(address, latitude, longitude) AS address_with_coordinates
FROM users_data;

-- To increase the readabiliy, it would be better for us to add commas and spaces (',
') between the columns
SELECT address || ', ' || latitude || ', ' || longitude AS address_with_coordinates
FROM users_data;
SELECT CONCAT(address,', ', latitude, ', ', longitude) AS address_with_coordinates # y
ou can have 'address with coordinates'
FROM users_data;

-- The LENGTH() function counts exactly how many characters are in a string (including
spaces).
-- We can nest functions! This combines the strings first, and THEN COUNTs the entire
result.
-- Here, we wrap LENGTH() around our concatenated string to see how long each combined
address is.
SELECT
    LENGTH(CONCAT(address, ', ', latitude, ', ', longitude)) AS Length_of_strings
FROM users_data;
```

```
SELECT
    LENGTH(address || ', ' || latitude || ', ' || longitude) AS Length_of_strings
FROM users_data;

-- UPPER() capitalizes all the characters within the string.
SELECT 'address', UPPER('address');

-- Here, we capitalize the actual data inside the address column for every row.
SELECT UPPER(address)
FROM users_data;

SELECT UPPER(address ||' ,' || latitude || ' ,' || longitude) AS 'address with coordin
ates'
FROM users_data;

-- LOWER() does the exact opposite, forcing all characters to lowercase.
SELECT LOWER(address ||' ,' || latitude || ' ,' || longitude) AS 'address with coordin
ates'
FROM users_data;

-- SUBSTR(string, start_position, length) extracts a specific chunk of text.
-- CRITICAL NOTE FOR STUDENTS: Unlike Python, where the index starts at 0, in SQL the
function is 1-based, meaning the first character is at index 1.
-- This grabs the first 5 characters of every address.
SELECT SUBSTR(address, 1, 5)
FROM users_data;

-- ==========================================
-- Now, we move onto DATA CLEANING: REPLACE & CAST (The "Money" Problem)
-- ==========================================
-- Next SQL built-in function I would like to cover is REPLACE(), which is usually bee
n used when we do data cleaning.
-- Let's raise a question: What is the total per capita income across all of customer
s.
-- Recall how to  preview our table data type. Using PRAGMA right?
-- WHERE per_capital_income suppose to be in numeric data type, but it is in National
Variable character datatype
PRAGMA table_info(users_data);

-- Let's take a look of what data are being stored within "Per Capita Income" column
(Where is has $ symbol within each data cell).
SELECT per_capita_income
FROM users_data;

-- To find  What is the total per capita income across all of customers.
-- Step A: We use REPLACE() to find every '$' and replace it with nothing (''), effect
ively deleting it.
SELECT per_capita_income, REPLACE(per_capita_income, '$', '')
FROM users_data;

-- It looks like a number now, but the database still thinks it is text.
-- Step B: We use CAST() to convert that cleaned text into a real INTEGER (or FLOAT/DE
CIMAL).
SELECT per_capita_income, CAST(REPLACE(per_capita_income, '$', '') AS REAL) # can be s
tring, float, decimal
FROM users_data;

-- Step C: Wait, our data also has commas (e.g., $29,278), some we need to clean the o
ur data column further before we cover the datatype of the column.
-- We can nest a second REPLACE() to strip out the commas as well!
```

```
SELECT per_capita_income, REPLACE(REPLACE(per_capita_income, '$', ''), ',', '')
FROM users_data;

-- Step D: Now we CAST that fully cleaned string into an REAL number.
SELECT per_capita_income, CAST(REPLACE(REPLACE(per_capita_income, '$', ''), ',', '') A
S REAL)
FROM users_data;

-- Step E: Finally, because it is a true number, we can safely perform math like SUM()
on it! To find the total per capita income of all of our customers.
SELECT per_capita_income, sum(CAST(REPLACE(REPLACE(per_capita_income, '$', ''), ',',
'') AS REAL))
FROM users_data;

-- TRIM() removes accidental blank spaces from the very beginning and end of a string.
SELECT ' hello ', TRIM(' hello ')

-- You can also tell TRIM to remove specific characters. both leading and trailing whi
tespace or other specified characters, like a trailing comma.
SELECT
    address || ',',
    TRIM(address || ',', ',') as clean_address
from users_data;

# Okay, this is enough of SQL built-in functions. (15 mins)

######## No need to mention
-- COALESCE() is a lifesaver for NULLs (missing) values.
-- It looks at the first column; if it is NULL, it replaces it with your backup text.
SELECT
    id,
    COALESCE(address, 'Address Unknown') AS Mailing_Address
FROM users_data;

# If you are summing up debt, and one person has NULL debt, the entire sum might turn
into NULL.
-- DANGER: Math with NULLs destroys your calculations. (NULL + 500 = NULL)
SELECT total_debt + 500 FROM users_data;

-- SAFE: COALESCE treats the NULL as a 0 before doing the math. (0 + 500 = 500)
SELECT COALESCE(total_debt, 0) + 500 FROM users_data;
```

`WHERE` clause

The `WHERE` clause is used to filter records. It is used to extract only those records that fulfill a specified condition.

The following operators can be used in the `WHERE` clause:

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=, exclamation and equal marks
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

```
-- Comparison Operators: =, >, <, >=, <=, <> or !=, BETWEEN, LIKE, IN

-- Let's query individuals who are strictly under the age of 20.
SELECT *
FROM users_data
WHERE current_age < 20;

-- Instead of comparing with a single value, we can also compare column by column
-- Let's say find aLL of retired customers within our system.
SELECT *
FROM users_data
WHERE current_age >= retirement_age;

-- Furthermore, we can combine multiple conditions using AND.
-- Find individuals under 30 who also make more than $80k a year.
SELECT *
FROM users_data
WHERE current_age < 30 AND yearly_income > 80000; # use <= as well

-- Recall aggregation function COUNT, we can use it to COUNT how many customers satisf
y the conditions.
SELECT COUNT(*)
FROM users_data
WHERE current_age < 30 AND yearly_income > 80000; # use <= as well

-- Instead of doing comparison, we can also pull out the exact information for one spe
cific customer using their ID.
SELECT *
FROM users_data
WHERE id = 1556; # Same as Student ID.

-- Let's move onto transactions data table.
# Recalled how to preview datatype of your table.
PRAGMA table_info(transactions_data);
-- Let's select all of the transactions from the transactions data table
SELECT *
FROM transactions_data;
```

Wildcards

A wildcard character is used to substitute one or more characters in a string. Wildcard characters are used with the `LIKE` operator. The `LIKE` operator is used in a `WHERE` clause to search for a specified pattern in a column.

Symbol	Description
%	Represents zero or more characters
_	Represents a single character
[]	Represents any single character within the brackets *
^	Represents any character not in the brackets *
-	Represents any single character within the specified range *
{}	Represents any escaped character **

```
-- So far, I have only shown you, how to filter with numeric data type columns, what a
bout text datatype columns?
-- For example, instead of search of ids can you pull out the transactions that happen
ed at 'Toronto'.
SELECT *
FROM transactions_data
WHERE merchant_city = "Toronto";
```

```
-- What happen if you don't remember how to spell Toronto, or you don't remember the e
xact name of a city?
-- What happens if you only know a city starts with 'T'?
-- We use the LIKE operator with the '%' wildcard.
-- The '%' acts as a placeholder for "zero or more characters of any kind".
-- The transactions that happens at the merchant cities, and the name of merchant citi
es start with T.
SELECT *
FROM transactions_data
WHERE merchant_city LIKE "T%"; # String start with T, where % is a wildcard symbol

-- String ends with 'T'. (e.g., Detroit, Frankfurt)
SELECT *
FROM transactions_data
WHERE merchant_city LIKE "%T"; # String end with T

-- String contains 'T' anywhere inside it. Note: LIKE is generally case-insensitive.
SELECT *
FROM transactions_data
WHERE merchant_city LIKE "%T%"; # String that contains T, non case-sensitive

-- LIKE is usually not case-sensitive, SQLite has a special function called GLOB.
-- GLOB is strictly case-sensitive.
-- Note: GLOB uses '*' as its wildcard instead of '%'.
SELECT merchant_city
FROM transactions_data
WHERE merchant_city GLOB 'T*'; -- try '*T' or '*T*'

SELECT merchant_city
FROM transactions_data
WHERE merchant_city GLOB '*t*';

-- Instead of only searching in one condition, we can also add more searching conditio
n by using AND logic operator, such as search the cities start with T but contains or
o.
-- Note: Order matters! 'oro', 'oor', and 'roo' will yield different results.
SELECT *
FROM transactions_data
WHERE merchant_city LIKE "T%" AND merchant_city LIKE "%oro%"

-- DANGEROUS QUERY:
-- I want to increase the complexity of the query by adding more conditions.
-- Let's say, We want to search cities starting with T and containing 'oro', located i
n EITHER Canada or NJ.
-- It is a dangerous query, also a common mistake made by people.
-- Result: It will return ANY transaction in NJ, completely ignoring the city name rul
es!
-- Because SQL processes 'AND' before 'OR' (like multiplication before addition in mat
h), it groups the first three conditions together, and leaves NJ completely by itself.
-- Select all of transactions at the merchant city starts with T and contain the patte
rn 'oro', located in Canada or get every single transactions in New Jersey.
SELECT *
FROM transactions_data
WHERE merchant_city LIKE "T%" AND merchant_city LIKE "%oro%" AND merchant_state = 'Can
ada' # Note case sensitive, cannot have CANADA
OR merchant_state = 'NJ';

-- SAFE QUERY (Using Parentheses): -- Parentheses force SQL to evaluate the 'OR' condi
tion as a single grouped requirement. Select all transactions where the merchant city
```

```
starts with 'T' and contains the pattern 'oro', and the location is either Canada or N
ew Jersey.
SELECT *
FROM transactions_data
WHERE merchant_city LIKE "T%" AND merchant_city LIKE "%oro%"
AND (merchant_state = 'Canada' or merchant_state = 'NJ');

# There is another way to search words
# The IN operator allows you to specify multiple values in a WHERE clause. search for
transactions happened at Toronto, Mexico, Vancouver.
SELECT merchant_city
FROM Transactions_data
WHERE merchant_city IN ('Toronto', 'Mexico', 'Vancouver');
# The IN operator is a shorthand for multiple OR conditions, It saves you from writin
g: WHERE city = 'Toronto' OR city = 'Mexico' OR city = 'Vancouver'
SELECT merchant_city
FROM transactions_data
WHERE merchant_city = 'Toronto' OR merchant_city = "Mexico" OR merchant_city = 'Vancou
ver';

-- You can also use IN with a subquery! Create a list by writing a subquery
-- This returns all records where the city matches a list generated by another query.
SELECT * FROM transactions_data
WHERE merchant_city IN (
    SELECT DISTINCT merchant_city
    FROM transactions_data
    LIMIT 10
);

-- Let's bring back our REPLACE() function to clean the amount column, as I want to in
troduce BETWEEN operator to you. Between provide a certain range of data, also used fo
r filtering.
-- Find the transactions amount between 10 and 12.
-- BETWEEN is inclusive, meaning it includes the exact numbers 10 and 12.
SELECT id, amount, CAST(REPLACE(amount, '$', '') AS DECIMAL) AS Transaction_amount
FROM transactions_data
WHERE Transaction_amount BETWEEN 10 AND 12;
-- This does the exact same thing using standard operators.
SELECT id, amount, CAST(REPLACE(amount, '$', '') AS DECIMAL) AS Transaction_amount
FROM transactions_data
WHERE 10 <= Transaction_amount AND Transaction_amount <= 12;

# Transaction amount between $10 and $12 for transaction id, 7475337, 7475353, 7475377
SELECT id, amount, CAST(REPLACE(amount, '$', '') AS DECIMAL) AS Transaction_amount
FROM transactions_data
WHERE Transaction_amount BETWEEN 10 AND 12
AND id IN (7475337, 7475353, 7475377);

-- Except filtering numbers, we can also use BETWEEN to filter text alphabetically! Fo
r example:
-- We select all merchant city with a city name alphabetically between A and B
SELECT merchant_city
FROM transactions_data
WHERE merchant_city BETWEEN 'A' AND 'B';
-- This selects all merchant cities starting with 'A' up to 'B' but excluding 'B'. Alt
hough, BETWEEN is inclusive. However, SQL reads text strictly alphabetically (lexicogr
aphically), exactly like a dictionary.
-- For instances, the word "Boston" comes after the single letter "B", Because "Bosto
n" is greater than "B", it falls completely outside the range! The only "B" city this
query would return is a city literally named just "B".
```

```
-- We can verify our query by using LIKE to search whether merchant_cities start with
B are in our list.
SELECT merchant_city
FROM transactions_dataWHERE merchant_city
BETWEEN 'A' AND 'B' AND merchant_city LIKE 'B%'; --  (as we don't have a merchant city
named 'B' Only.)

-- SOLUTION 1: Push the boundary to the next letter
SELECT merchant_city
FROM transactions_data
WHERE merchant_city BETWEEN 'A' AND 'C';
-- SOLUTION 2: Push the boundary to the next letter
-- This gets all 'A' and 'B' cities, stopping right before 'C' cities.
SELECT address, merchant_city
FROM transactions_data
WHERE merchant_city >= 'A' AND merchant_city < 'C';
-- SOLUTION 3: The "Wildcard" approach (Usually the easiest for beginners)
-- Using the LIKE operator explicitly tells SQL exactly what starting letters you wan
t.
SELECT address, merchant_city
FROM transactions_data
WHERE merchant_city LIKE 'A%' OR merchant_city LIKE 'B%';

-- BETWEEEN can also be used by Filtering Date Ranges (YYYY-MM-DD format is best). Le
t's find all of the transactions happened on July of 2010.
SELECT *
FROM transactions_data
WHERE date BETWEEN '2010-07-01' AND '2010-07-31'; -- (35 mins)

-- Let's get back to LIKE operator, I want to introduce you another wildcards, which i
s the underscore '_'. Underscore wildcard stands for EXACTLY ONE character.
-- This returns all transactions where the city starts with ANY single character, foll
owed by 'oronto'. E.g., 'Toronto' or 'Moronto'.
-- Note for students: '__oronto' (with two underscores) will NOT match 'Toronto' becau
se it tells the database to look for an 8-letter word!
SELECT *
FROM transactions_data
WHERE merchant_city LIKE '_oronto';

-- This returns all transactions where the city starts with 'T', is followed by EXACTL
Y 4 unknown characters, and ends with 'to'. E.g., 'Toronto' fits perfectly (T + oron +
to).
SELECT *
FROM transactions_data
WHERE merchant_city LIKE 'T____to';

######################################## No need to mention maybe
-- GLOB is a powerful tool in SQLite that allows you to use Unix-style pattern matchin
g.
-- CRITICAL REMINDER: GLOB is case-sensitive, and it uses '*' instead of '%' for wildc
ards.
-- Using brackets [] allows you to match against a specific list of characters.
-- Find the transactions where the merchant city starts with exactly 'A', 'B', 'C', or
'T'.
SELECT merchant_city
FROM transactions_data
WHERE merchant_city GLOB '[ABCT]*';

-- You can also use a hyphen inside the brackets to define a range!
-- Find the transactions where the merchant city starts with any uppercase letter from
```

```
'A' through 'Z'.
SELECT merchant_city
FROM transactions_data
WHERE merchant_city GLOB '[A-Z]*';

-- Using the caret symbol '^' inside the brackets NEGATES the match.
-- Find the transactions where the merchant city does NOT start with uppercase 'A' thr
ough 'X'
-- (This effectively leaves 'Y' and 'Z', as well as lowercase letters and numbers).
SELECT merchant_city
FROM transactions_data
WHERE merchant_city GLOB '[^A-X]*';
```

```
###### No need to mention this section maybe, they are DML Languages, briefly
-- The UPDATE statement is used to modify existing records in a table. You can change
one column or multiple columns at the same time.
-- DANGER ZONE: If you forget to include the WHERE clause, the database will apply the
change to EVERY SINGLE ROW in the entire table!
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;

UPDATE users_data
SET address = '123 New Toronto Ave', credit_score = 780
WHERE id = 1556;

-- ==========================================
-- 2. ADDING NEW DATA: METHOD 1 (The "Safe" Way)
-- ==========================================
-- In this method, you explicitly specify both the column names and the values. BEST P
RACTICE: This is the safest and most professional way to write an INSERT.
-- Even if the database administrator adds a new column to the table later, this code
will still run perfectly because you are telling SQL exactly where to put the data.
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
INSERT INTO transactions_data (id, date, client_id, amount, merchant_city)
VALUES (900001, '2026-02-23 08:30:00', 1556, '$4.50', 'Toronto');

-- ==========================================
-- ADDING NEW DATA: METHOD 2 (The "Shortcut" Way)
-- ==========================================
-- If you are adding values for ALL the columns in the table, you can skip writing the
column names.
-- However, you MUST ensure that the values you provide are in the exact same order
-- as the columns defined in the table's schema. (If you put a date where an amount sh
ould go, the query will fail).
INSERT INTO table_name
VALUES (value1, value2, value3);
INSERT INTO users_data
VALUES (9999, 25, 65, 2001, 5, 'Female', '456 Data Street', 43.65, -79.38, '$60000', 6
0000.0, 0.0, 750, 1);

-- ==========================================
-- REMOVING DATA (The DELETE Command)
-- ==========================================
-- The DELETE statement is used to permanently remove rows from a table.
-- DANGER ZONE: Just like the UPDATE command, the WHERE clause is your safety net. If
you run `DELETE FROM table_name;` without a condition, it will wipe out all the data!
DELETE FROM table_name
```

```
WHERE condition;

DELETE FROM users_data
WHERE id = 9999
```

The SQL `ORDER BY`

```
-- Next, we want to cover the ORDER BY clause, which is used to sort your result set.
The ORDER BY keyword is used to sort the result-set in ascending or descending order.
Also, the ORDER BY keyword sorts the records in ascending order by default. To sort th
e records in descending order, use the DESC keyword.
-- So technically, by default, if you don't specify a direction, it always sorts in AS
CENDING order, from lowest to highest value.
SELECT *
FROM users_data
ORDER BY id; -- same as ORDER BY id ASC

-- To reverse the order (highest to lowest, or 10, 9, 8...), you must explicitly add t
he DESC (Descending) keyword at the end of the statement.
SELECT *
FROM users_data
ORDER BY id DESC;

-- You can sort by more than one column! SQL reads your instructions from left to righ
t.
-- The Query selects all transactions and sorts them primarily by "merchant_city".
-- If multiple rows happen to have the exact same city, it then uses the "transaction
id" to tie-break and sort them.
SELECT *
FROM transactions_data
ORDER BY merchant_city;

SELECT *
FROM transactions_data
ORDER BY merchant_city, id;

-- You can even mix and match the sorting directions for each column! This orders the
cities alphabetically (Ascending), but within each specific city, it lists the IDs fro
m highest to lowest (Descending).
SELECT *
FROM transactions_data
ORDER BY merchant_city ASC, id DESC;
```

The SQL GROUP BY Statement

The `GROUP BY` statement groups rows that have the same values into summary rows, like "find the number of transaction in each city". The `GROUP BY` statement is often used with aggregate functions (`COUNT()`, `SUM()`, `AVG()`) to group the result-set by one or more columns.

```
-- ==========================================
-- THE SQL BLUEPRINT
-- ==========================================
-- When building complex queries, you must write the clauses in this exact order:
SELECT column_name(s)
FROM table_name
WHERE condition              -- Filters raw rows BEFORE grouping
GROUP BY column_name(s)      -- Squishes the data into categories
ORDER BY column_name(s);     -- Sorts the final output
```

```
-- ==========================================
-- INTRODUCTION TO GROUP BY
-- ==========================================
-- Unlike aggregate functions (like COUNT, SUM, AVG) squish your entire table into one
row.
SELECT COUNT(merchant_city)
FROM transactions_data;

-- But what happen if we want to count how many transactions happened FOR EACH city?
-- GROUP BY tells the database to split the data into buckets (cities) first, and then
count what is inside each bucket.
-- Let's find the total number of transactions that occurred in each city.
SELECT merchant_city, COUNT(merchant_city) AS total_transactions
FROM transactions_data
GROUP BY merchant_city;

-- Combining this with what we learned about ORDER BY, we can rank our result!
-- Let's see which cities have the highest volume of transactions.
SELECT merchant_city, COUNT(merchant_city) AS total_transactions
FROM transactions_data
GROUP BY merchant_city
ORDER BY total_transactions DESC;

-- ==========================================
-- GROUP BY WITH SUM() AND DATA CLEANING
-- ==========================================
-- Which city's residents spend the most money in total?
-- Let's bring back our REPLACE() and CAST() tricks from earlier, so that SUM() can ac
tually do the math!
SELECT merchant_city, SUM(CAST(REPLACE(amount, '$', '') AS DECIMAL)) AS total_amount
FROM transactions_data
GROUP BY merchant_city
ORDER BY total_amount DESC;

-- We can apply the exact same logic to find our highest-spending individuals!
-- Which specific customer spent the most money overall?
SELECT client_id, SUM(CAST(REPLACE(amount, '$', '') AS DECIMAL)) AS total_amounts
FROM transactions_data
GROUP BY client_id
ORDER BY total_amounts DESC;
```

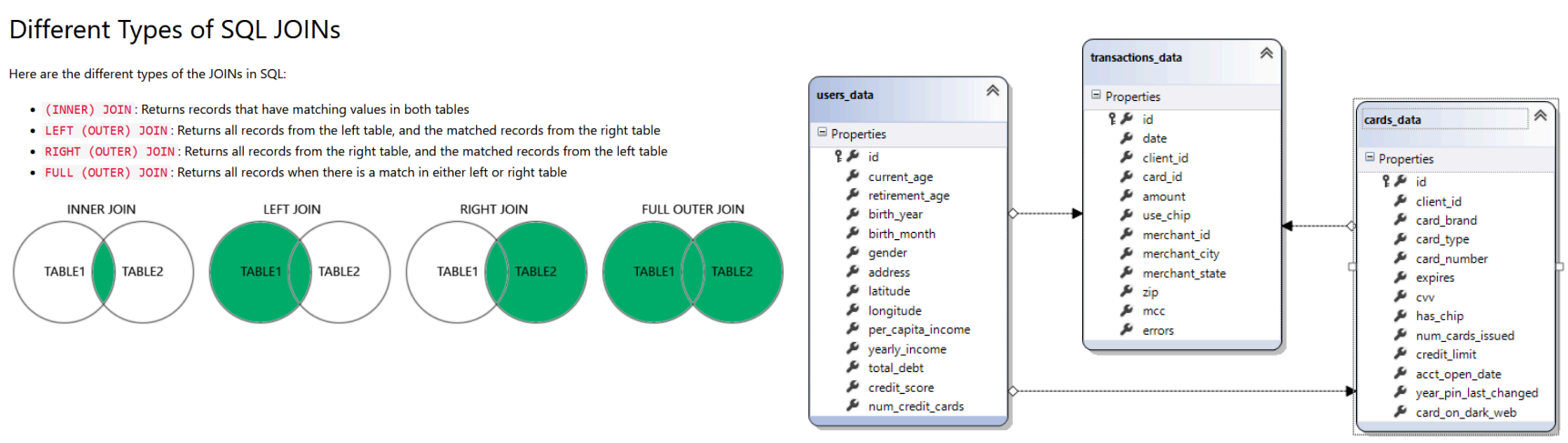
The `HAVING` clause was added to SQL because the `WHERE` keyword cannot be used with aggregate functions.

```
-- ==========================================
-- THE SQL BLUEPRINT
-- ==========================================
-- When building complex queries, you must write the clauses in this exact order:
SELECT column_name(s)
FROM table_name
WHERE condition              -- Filters raw rows BEFORE grouping
GROUP BY column_name(s)      -- Squishes the data into categories
HAVING condition             -- Filters the summary metrics AFTER grouping
ORDER BY column_name(s);     -- Sorts the final output

-- ==========================================
-- THE 'HAVING' CLAUSE (Filtering Groups)
-- ==========================================
-- Let's say we want to find cities with very low activity (less than 5 transactions).
-- We CANNOT use the WHERE keyword to filter this, because WHERE looks at raw rows bef
ore the counting even happens.
```

```
-- HAVING is the special filter you use specifically for aggregate functions.
SELECT merchant_city, COUNT(merchant_city) AS city_numbers
FROM transactions_data
GROUP BY merchant_city
HAVING city_numbers < 5           -- Filters the calculated counts
ORDER BY city_numbers DESC;

-- ==========================================
-- THE "ERROR" DEMONSTRATION
-- ==========================================
-- THIS WILL GIVE YOU A SQL ERROR!
-- Why? Because of the "Order of Execution". SQL reads the WHERE clause before it read
s the GROUP BY clause. It tries to filter by "numbers", but "numbers" hasn't been calc
ulated yet!
SELECT merchant_city, COUNT(merchant_city) AS numbers
FROM transactions_data
WHERE numbers < 5            -- ERROR: Aggregate functions cannot be used in WHERE
GROUP BY merchant_city
ORDER BY numbers DESC;
```

`JOIN` and `ON`

Now, we are going to learn one of the most important clauses in SQL, which is `JOINs`. A `JOIN` clause is used to combine rows from two or more tables, based on a related column between them.




```
-- Think of it as the intersection of two circles in the diagram.
-- Syntax Blueprint:
-- SELECT column_name(s)
-- FROM table1
-- INNER JOIN table2 ON table1.column_name = table2.column_name;

-- Let's look at our two main tables separately first. # Recall users_data and cards_d
ata data tables
SELECT * FROM users_data;
SELECT * FROM cards_data;

# We inner joint the users data table with the cards data table by using client id fie
ld from both tables. As we mention, INNER JOIN keyword returns only rows with a match
in both tables. Which means that if you have a users (client) with no client_id, or wi
th a client_id that is not present in the cards data table or he/she does not own a ca
rd, that record would not be returned in the result.
-- Now, let's join them! We link the users table with the cards table using the client
ID.
# Note, this query will give first users data table columns/fields first, then cards d
ata table, as we read from users data table first.
SELECT *
```

```
FROM users_data
INNER JOIN cards_data ON users_data.id = cards_data.client_id

-- recall, how do we find the number of rows that a table has? Let's COOUNT how many r
ows are there?
SELECT COUNT(*) FROM cards_data;

-- To see whether each client has at least one credit or debit card, we can also count
our JOIN table
-- Because the INNER JOIN keyword selects records that have matching values in BOTH ta
bles only, if number of rows matches with cards data table, then definitely each clien
t has at least one credit or debit card.
SELECT COUNT(*)
FROM users_data
INNER JOIN cards_data ON users_data.id = cards_data.client_id;

-- Typing out full table names gets exhausting. We can use 'AS' to give tables short n
icknames.
SELECT *
FROM users_data AS ud
INNER JOIN cards_data AS cd ON ud.id = cd.client_id

-- Pro-Tip: The 'AS' keyword is actually optional for table renames! You can write wit
hout AS to rename tables, just put a space and the nickname.
SELECT *
FROM users_data ud
INNER JOIN cards_data cd ON ud.id = cd.client_id

# JOIN and INNER JOIN will return the same result. INNER is the default join type for
JOIN, so when you write JOIN, you are actually writing INNER JOIN.
SELECT *
FROM users_data ud
JOIN cards_data cd ON ud.id = cd.client_id

-- Furthermore, we see there is a number of credit cards that each client holds colum
n, we can do a cross check between two tables.
-- Let's add up the 'num_credit_cards' column across every single user.
-- It should tell us the total number of credit cards our customers claim to have base
d on their demographic profiles.
SELECT SUM(num_credit_cards)
FROM users_data;

-- We then count the exact number of unique card IDs in our product table.
-- It tells us the actual, physical number of credit cards the bank has issued and is
actively tracking in the system.
SELECT COUNT(DISTINCT id)
FROM cards_data;
# This is also a cross check between two tables

# Let's get back to JOINs.
SELECT *
FROM users_data ud
INNER JOIN cards_data cd ON ud.id = cd.client_id
-- Note, you need to specify the table name when you callout or select specified colum
n names, if the columns are not present in both tables.
-- It is like VLOOKUP. We have users and cards data, but we don't know the customer's
card number—only their ID. We need to fetch the card number from the other table.
-- If you don't want every single column, you must specify which table a column comes
from by using its alias (e.g., ud.address).
SELECT ud.id AS client_id, cd.id AS card_id, ud.current_age, ud.birth_year, ud.addres
```

```
s, ud.per_capita_income, cd.card_type, cd.card_number
FROM users_data ud
JOIN cards_data cd ON ud.id = cd.client_id;

-- Now we have learned INNER JOIN, let's try to join the entire three tables together
to see the full picture of three datasets.
-- CONCEPTUAL CHECK: Why do we join the transactions table using 'card_id' instead of
'client_id'?
-- Imagine me (Yi-Ming) has 2 cards (Debit and Visa), and buys 1 coffee today. If we j
oin the transaction directly to the client_id, the database sees 2 cards for Yi-Ming a
nd will duplicate the coffee purchase, making it look like I bought a coffee on BOTH c
ards! By linking the transaction directly to the specific CARD used, we prevent this d
uplication. (BREAK NOW)
SELECT *
FROM users_data ud
JOIN cards_data cd ON ud.id = cd.client_id      -- Link User to their Cards
JOIN transactions_data td ON cd.id = td.card_id; -- Link Card to its SPECIFIC Transact
ions

# If we have time, we can run following two queries:
SELECT COUNT(*)
FROM users_data ud
JOIN cards_data cd ON ud.id = cd.client_id      -- Link User to their Cards
JOIN transactions_data td ON cd.id = td.card_id; -- Link Card to its SPECIFIC Transact
ions
SELECT COUNT(*)
FROM transactions_data; -- number matches, no missing cards, every transaction matches
with cooresponding credit card  in our system.

-- The LEFT JOIN keyword returns ALL records from the left table, and the matched reco
rds from the right table. If there is no match, the right side just shows up as NULL.
The result is 0 records from the right side, if there is no match.
SELECT *
FROM transactions_data t
LEFT JOIN cards_data c ON c.id = t.card_id
GROUP BY card_id;

-- Let's combine JOINs with our GROUP BY skills! We want to see the total amount spent
on each specific card. (Notice we use SUM to add up the money, and COUNT to find the n
umber of swipes).
# Without SUM Windows functions: (problem) There are 1,000 transactions in Tokyo. Whic
h "amount" should it show? The $5 coffee? The $500 dinner?
SELECT t.client_id, t.card_id, SUM(CAST(REPLACE(t.amount, "$", "") AS DECIMAL)) AS tot
al_amount, c.card_number, COUNT(c.card_number) AS Number_of_transactions
FROM transactions_data t
LEFT JOIN cards_data c ON c.id = t.card_id
GROUP BY c.card_number;

# Order your new 'Number_of_transactions' columns in descending order.
SELECT t.client_id, t.card_id, SUM(CAST(REPLACE(t.amount, "$", "") AS DECIMAL)) AS tot
al_amount, c.card_number, COUNT(c.card_number) AS Number_of_transactions
FROM transactions_data t
LEFT JOIN cards_data c ON c.id = t.card_id
GROUP BY c.card_number
ORDER BY Number_of_transactions DESC;

-- A RIGHT JOIN is the exact opposite of a LEFT JOIN. It keeps everything in the right
table (cards_data) regardless of matches. And then finding any matches from transactio
ns data table.
SELECT t.client_id, t.card_id, SUM(CAST(REPLACE(t.amount, "$", "") AS DECIMAL)) AS tot
```

```
al_amount, c.card_number, COUNT(c.card_number) AS Number_of_transactions
FROM transactions_data t
RIGHT JOIN cards_data c ON c.id = t.card_id
GROUP BY c.card_number
ORDER BY Number_of_transactions DESC;

################ No need to include the following ####################################
#
###### Let's aviod to use transactions data for joins as the query takes too much time
due to there are more than 13 million rows within the data table.
# If these two queries return the exact same number, it means every single card in you
r database belongs to a valid user.

-- A FULL OUTER JOIN combines the results of both LEFT and RIGHT joins. It returns all
matching records from both tables, PLUS all the unmatched rows from both sides.
-- This is an excellent way to audit your data for missing links!
-- If the COUNT of a LEFT JOIN and a FULL OUTER JOIN return the exact same number, it
proves that every single card in your database belongs to a valid, existing user.
SELECT count(*)
FROM users_data u
LEFT JOIN cards_data c ON u.id = c.client_id;

SELECT count(*)
FROM users_data u
FULL OUTER JOIN cards_data c ON u.id = c.client_id;


SELECT *
FROM transactions_data t
LEFT JOIN users_data u ON u.id = t.client_id
ORDER BY u.id;

SELECT *
FROM transactions_data t
FULL OUTER JOIN users_data u ON u.id = t.client_id
ORDER BY u.id;
SELECT
    date,
    amount,
    FIRST_VALUE(amount) OVER (
        PARTITION BY client_id   -- Group by User
        ORDER BY date            -- Sort by Date
    ) AS First_Purchase_Amount
FROM transactions_data;

# Professional Data Engineer Writing
-- Step 1: Create a temporary "virtual table" (CTE) that ranks the purchases
WITH RankedPurchases AS (
    SELECT
        client_id,
        date,
        amount,
        ROW_NUMBER() OVER (         -- This assigns a 1, 2, 3... to each purchase base
d on the date
            PARTITION BY client_id
            ORDER BY date
        ) AS purchase_rank
    FROM transactions_data
)
SELECT -- Step 2: Query that virtual table and filter for ONLY rank 1
```

```
    client_id,
    date AS First_Purchase_Date,
    amount AS First_Purchase_Amount
FROM RankedPurchases
WHERE purchase_rank = 1;
```

Full Example:

```
--  find individual users (over age 35) who personally own multiple cards, (id, curren
t age, ...) give student some time to think about it. I will give you 5 mins on how to
write this SQL query.
SELECT
    u.id,
    u.current_age,
    COUNT(c.card_number) AS total_cards
FROM users_data u
JOIN cards_data c ON u.id = c.client_id
WHERE u.current_age > 35
GROUP BY u.id
HAVING total_cards > 1
ORDER BY total_cards DESC;

# If you can understand this single block of code, you have officially learned SQL. Th
is is what we call a Master Query because it brings together everything we have learne
d today into one real-world business scenario. Let's say our Marketing department want
s to run a premium credit card campaign. They tell you: 'We want a list of established
adults—over the age of 35—who already have a history of managing multiple credit cards
with us.'

-- You may think the SQL query process is from top-to-bottom, exactly as you write it:
starting with SELECT and ending with LIMIT. Many SQL learning website, or textbooks ma
y not teach you this. The database engine actually processes the query almost complete
ly backward! This is one of the most important concepts for beginners to grasp. Unders
tanding the difference between the Lexical/writing Order (how we type it) and the Logi
cal Order (how the engine executes it) explains why certain errors happen—like why you
can't use an alias defined in the SELECT clause inside your WHERE clause.

Explain below stuff then here.
--- Let's break down exactly how the database reads this:
-- FROM and JOIN: First, we grab our users_data table and join it to our cards_data ta
ble. Notice we gave them nicknames, u and c, just to save us from typing the full tabl
e names over and over.
-- WHERE u.current_age > 35: Before the database does any math, we filter the raw dat
a. We immediately kick out anyone who is 35 or younger. They don't even make it to the
next step.
-- GROUP BY: Now we take all those remaining adults and collapse them into their own u
nique buckets using their id.
-- HAVING total_cards > 1: This is the most important lesson of the afternoon! Why did
n't we use WHERE here? Because WHERE filters raw data. HAVING filters summarized data.
-- ORDER BY: Finally, we take our summarized, filtered list and sort it from highest t
o lowest, handing Marketing a perfectly clean list of target customers."
```

You may think the SQL query process is top-to-bottom, exactly as you write it: starting with `SELECT` and ending with `LIMIT`. Many SQL learning website, or textbooks may not teach you this. The database engine actually processes the query almost completely backward! This is one of the most important concepts for beginners to grasp. Understanding the difference between the **Lexical Order** (how we type it) and the **Logical Order** (how the engine executes it) explains why certain errors happen—like why you can't use an alias defined in the `SELECT` clause inside your `WHERE` clause.

Process order: (FROM, JOIN, ON) → (WHERE) → (GROUP BY) → (HAVING) → (SELECT) → (DISTINCT) → (ORDER BY) → (OFFSET) → (LIMIT)

1. `FROM`, `JOIN`, `ON` **(Gathering the Data):** The database first identifies the base tables. If there are joins, it uses the `ON` conditions to merge the tables together into one massive, temporary working dataset
2. `WHERE` **(The First Filter):** Before doing any heavy lifting or math, the database filters out the rows that don't meet the basic criteria. (e.g., throwing out anyone under 35).
3. `GROUP BY` **(Organizing):** The remaining rows are sorted into distinct groups based on the columns specified.
4. `HAVING` **(The Second Filter):** Now that the data is grouped, the database filters the *groups* (e.g., throwing out any card numbers that don't have a count greater than 1).
5. `SELECT` **(Picking the Columns):** Finally, the database looks at the `SELECT` clause. It grabs the specific columns requested and calculates any math or aliases (like `AS TotalScore`). *This is why you cannot use a `SELECT` alias in a `WHERE` clause—the `WHERE` clause happened in Step 2, long before the alias was created!*
6. `DISTINCT` **(Duplicating):** If requested, the engine sweeps through the selected columns and removes any duplicate rows.
7. `ORDER BY` **(Sorting):** The final dataset is sorted based on the specified columns. (Because `SELECT` has already happened, you *can* use your `SELECT` aliases here).
8. `OFFSET` **&** `LIMIT` **(Paging):** As the very last step, the database skips the requested number of rows (`OFFSET`) and hands back the exact number of rows asked for (`LIMIT`).

```
-- ==========================================
-- THE CASE STATEMENT (IF/THEN Logic)
-- ==========================================
-- A CASE statement allows us to create brand new categories based on our existing dat
a. It evaluates conditions row by row, stopping at the first one that is TRUE. sort of
like functions in Python.
-- CASE statement:
CASE
    WHEN Condition1 is TRUE THEN result1
    WHEN Condition2 is TRUE THEN result2
    ELSE result3
END
-- We want to categorize every transaction as a Small, Medium, or Large trade. We can
mark transaction amount lower than 10 dollar as small trade, transaction amount higher
than 100 dollar as large trade...
SELECT
        client_id,
        amount,
    -- 1. First, we clean the text and turn it into a number
    CAST(REPLACE(amount, '$', '') AS DECIMAL) AS transaction_amount,
    -- 2. Then, we use CASE to categorize that number into buckets
    -- NOTE: We must repeat the math formula here because SQL hasn't fully registered
the "transaction_amount" alias yet!
    CASE
        WHEN CAST(REPLACE(amount, '$', '') AS DECIMAL) < 10 THEN 'Small Trade'
        WHEN CAST(REPLACE(amount, '$', '') AS DECIMAL) > 100 THEN 'Large Trade'
        ELSE 'Medium Trade'
    END AS Trade_Size
FROM transactions_data;

# we can also try to categorize our client credit plan by their credit card limit usin
g Case statement
-- Imagine you are working on the Marketing team, and you want to assign a status tier
to every credit card in the system. We use CASE WHEN limit > 20000 THEN 'Platinum'. Th
e database reads from top to bottom. If the card isn't Platinum, it checks if it's Gol
d. If it's neither, it catches everything else in our ELSE 'Standard' bucket.
SELECT id, credit_limit,
```

```
  CASE
    WHEN CAST(REPLACE(credit_limit, '$', '') AS DECIMAL) > 20000 THEN 'Platinum'
    WHEN CAST(REPLACE(credit_limit, '$', '') AS DECIMAL) > 10000 THEN 'Gold'
    ELSE 'Standard'
  END AS Card_Tier
FROM cards_data;

-- Using IIF() in SQL Server
-- Syntax: IF(condition, value_if_true, value_if_false)
# If you want to use an "IF" statement inside a SELECT query to change row values, you
actually use a function called IIF() (in SQL Server/Access) or IF() (in MySQL). Becaus
e you have three tiers (Platinum, Gold, Standard), you have to "nest" the IF statement
s inside each other, exactly like you would in Microsoft Excel.
SELECT
    id,
    credit_limit,
    IF(credit_limit > 20000, 'Platinum',
        IF(credit_limit > 10000, 'Gold', 'Standard')
    ) AS Card_Tier
FROM cards_data;

# Readability & Nesting: IF() statement: Great for a simple binary choice (e.g., True/
False, Yes/No). However, if you have 5 different credit card tiers, you have to nest 5
IF() statements inside each other. It becomes a nightmare to read and troubleshoot (of
ten called "Spaghetti Code"). CASE: Designed specifically to handle multiple condition
s. It reads cleanly from top to bottom (WHEN x THEN y).
# The "Other" IF Statement (Control Flow): There is a completely different type of IF
statement in SQL used for Control Flow (scripting). It is not used inside a query; it
is used to control whicTThe query runs. Example: "IF today is Monday, run the update s
cript. ELSE, run the backup script."
# The Golden Rule for Your Students: "Always use CASE." Even for simple True/False sce
narios, training yourself to use CASE ensures your code is readable, professional, and
guaranteed to run on any database system in the world.
```

Summary

```
SELECT
    DISTINCT [column_names_with AS [nickname/rename]
FROM
    [table_name_or_database_name]
JOIN [other_table_name] ON []
WHERE
    [conditions for_filltering numbers/strings/categories, uses Mathematical calculati
on symbols]
GROUP BY
    [column_name]
HAVING
    [additional conditions follow by groupby, constants, strings, Window Functions(av
g,...), column_name]
ORDER BY
    [column_name1 DESC, column_name2 ASC column3 DESC]
LIMIT 3 OFFSET 2; [start_from_third_row_and_limit_to_show_3_rows_only]
```

Process order: (FROM, JOIN, ON) → (WHERE) → (GROUP BY) → (HAVING) → (SELECT) → (DISTINCT) → (ORDER BY) → (OFFSET) → (LIMIT)

1. COUNT(*) and COUNT(1) differences?

This is one of the most common questions students ask, and it is surrounded by one of the biggest myths in the SQL world! Here is the straightforward answer you can share with your class: **Functionally and for performance,**

there is absolutely zero difference between `COUNT(*)` and `COUNT(1)` in modern databases.

Exercise (if we have time, we can go onto HackerRank or LeetCode to practice with students)

https://www.hackerrank.com/domains/sql?filters%5Bstatus%5D%5B%5D=unsolved&badge_type=sql

https://leetcode.com/studyplan/top-sql-50/